

Insurance Policy Intelligence & Agentic RAG Assistant

Software Requirements Specification (SRS)

Document Status: Draft — Version 0.1

Project Type: Industry-Oriented Large Language Model & Agentic AI System

Domain: Insurance

Primary Technology Area: Retrieval-Augmented Generation (RAG), Agentic AI, Evidence Grounding

Intended Audience: Project Team, Faculty Evaluators, Developers, Reviewers, Technical Interviewers

1. Introduction

1.1 Purpose

This document specifies the requirements for an **Insurance Policy Intelligence & Agentic RAG Assistant**, an AI-powered decision-support system designed to help users understand, retrieve, compare, and reason over information contained in insurance policy documents.

Insurance products are typically described through lengthy and heterogeneous documents such as policy wordings, brochures, terms and conditions, eligibility criteria, exclusions, benefits, waiting-period clauses, claim procedures, and product descriptions.

Finding relevant information manually across these documents can be time-consuming and may result in inconsistent interpretation.

The proposed system will use Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), semantic retrieval, agentic workflows, and evidence-grounding mechanisms to provide answers that remain connected to the available insurance documentation.

The system is intended to function as an **information and decision-support assistant**, not as an autonomous insurance underwriting, legal, medical, or financial decision-maker.

1.2 Problem Statement

A conventional document search system requires users to manually identify relevant documents and sections before interpreting the information.

A conventional LLM-based chatbot introduces another problem: the model may generate information that is plausible but unsupported by the underlying insurance documents.

The system therefore needs to solve two related problems:

1. **Information retrieval:** Identify the relevant evidence from a large collection of insurance documents.
2. **Evidence-grounded reasoning:** Generate useful responses while preventing unsupported claims from being presented as facts.

The system should additionally support complex queries requiring information from multiple sections or products.

Example:

"Compare these three health insurance policies for hospitalization coverage, waiting period, exclusions, and eligibility for a 45-year-old applicant."

This cannot always be answered reliably through a single retrieval-generation step.

2. Project Objectives

The primary objectives are:

O1 — Document Intelligence

Convert heterogeneous insurance documents into a searchable, structured knowledge source.

O2 — Semantic Retrieval

Retrieve relevant policy information based on the meaning of the user's query rather than only exact keyword matches.

O3 — Contextual Question Answering

Generate answers using retrieved insurance evidence as the primary knowledge source.

O4 — Agentic Query Processing

Decompose complex insurance queries into smaller information-gathering and reasoning tasks.

O5 — Product Comparison

Compare multiple insurance products according to user-specified criteria.

O6 — Evidence Grounding

Associate generated claims with supporting source passages.

O7 — Hallucination Control

Prevent unsupported information from being presented as policy fact.

O8 — Evidence-Aware Abstention

When sufficient evidence is unavailable, the system should explicitly communicate the limitation and request clarification or additional information rather than fabricate an answer.

O9 — Measurable Evaluation

Evaluate the system against a simpler RAG baseline using objective technical metrics.

3. Scope

3.1 In Scope

The system will support:

- Insurance document ingestion
- PDF/text processing
- Document cleaning
- Metadata extraction
- Structure-aware chunking
- Semantic embeddings
- Vector-based retrieval
- Optional hybrid retrieval
- Retrieval reranking
- Query classification
- Multi-step query processing
- Policy information extraction
- Coverage analysis
- Eligibility analysis
- Exclusion analysis
- Waiting-period analysis
- Product comparison
- Evidence-grounded response generation
- Source citations
- Unsupported-claim detection
- Clarification questions
- Refusal/abstention when evidence is insufficient
- Evaluation against a baseline RAG system

- API-based access
 - User-facing interface
 - Logging and monitoring
 - Containerized deployment
-

3.2 Out of Scope

The system will not:

- Issue actual insurance policies
- Approve or reject real insurance applications
- Calculate legally binding premiums
- Provide legally binding advice
- Provide medical diagnosis
- Replace licensed insurance professionals
- Access confidential insurer databases
- Execute real financial transactions
- Automatically purchase insurance
- Make autonomous underwriting decisions

Public or synthetic documents will be used during development unless authorized data is explicitly provided.

4. Intended Users

4.1 End User

A person who wants to understand or compare insurance policies.

Example queries:

- "Does this policy cover hospitalization?"
- "What are the exclusions?"
- "What is the waiting period?"
- "Compare Policy A and Policy B."
- "Which policy satisfies my specified requirements?"

4.2 Domain Reviewer

A person who evaluates whether the system's responses and citations correctly represent the underlying policy documents.

4.3 System Administrator

Responsible for:

- Document management
- System configuration
- Monitoring
- Evaluation datasets
- Model configuration
- Access control

4.4 Developer / Maintainer

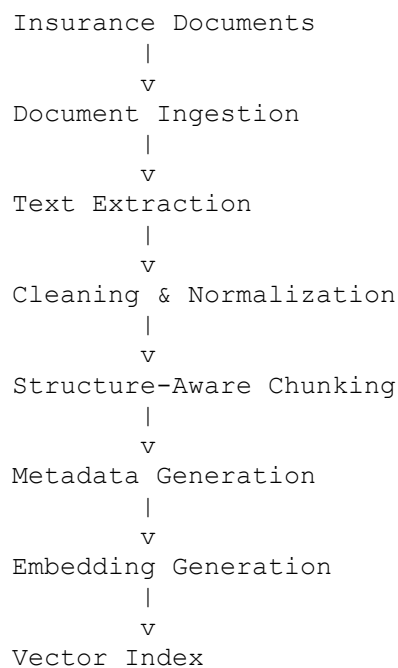
Responsible for:

- Application development
- Retrieval pipeline
- Agent workflows
- Grounding mechanisms
- API services
- Deployment
- Testing

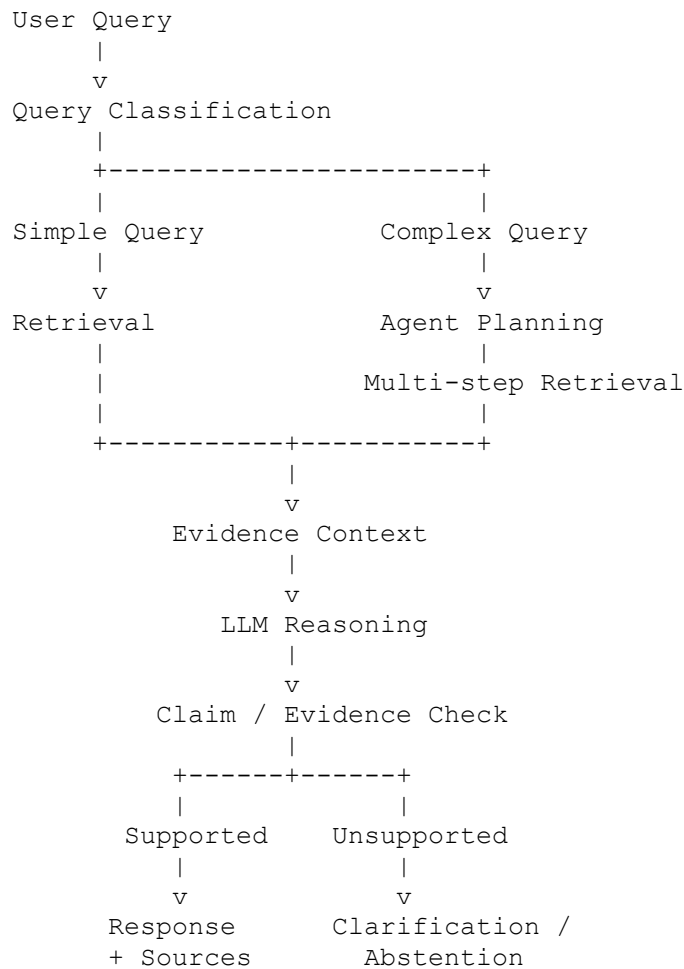
5. High-Level System Concept

The proposed system consists of two major operational flows.

5.1 Offline Knowledge Preparation



5.2 Online Query Processing



6. Functional Requirements

FR-001 — Document Ingestion

The system shall allow authorized users or administrators to ingest supported insurance documents.

Supported initial format:

- PDF

Future formats may include:

- DOCX
- HTML
- TXT

The system shall assign a unique identifier to every ingested document.

FR-002 — Document Metadata

Each document shall maintain metadata including, where available:

- Document ID
- Document name
- Product name
- Insurance category
- Version
- Publication/effective date
- Source
- Page count
- Ingestion timestamp

FR-003 — Text Extraction

The system shall extract machine-readable text from uploaded documents.

The extraction process shall preserve page-level information wherever possible.

FR-004 — Structure-Aware Chunking

The system shall divide documents into retrieval units while attempting to preserve semantic and structural boundaries.

A chunk should preferably retain information such as:

- Document ID
- Page number
- Section
- Subsection
- Product
- Chunk ID

The system shall avoid unnecessary fragmentation of clauses whose meaning depends on surrounding text.

FR-005 — Embedding Generation

The system shall generate vector representations for searchable document chunks.

The embedding model shall be configurable.

FR-006 — Vector Storage

The system shall store document embeddings in a vector-search system.

The vector index shall support:

- Similarity search
 - Top-k retrieval
 - Metadata filtering
 - Document-level identification
-

FR-007 — Query Classification

The system shall classify incoming queries into appropriate processing categories.

Initial categories may include:

- Policy information query
 - Coverage query
 - Eligibility query
 - Exclusion query
 - Waiting-period query
 - Product comparison
 - Recommendation request
 - Multi-step/complex query
 - Unsupported/out-of-domain query
-

FR-008 — Relevant Evidence Retrieval

The system shall retrieve passages relevant to the user's query.

The retrieval subsystem may combine:

- Semantic retrieval
- Keyword retrieval
- Metadata filtering
- Reranking

The final retrieved context shall be supplied to the reasoning/generation component.

FR-009 — Context Construction

The system shall construct a bounded context for the LLM.

The context shall contain relevant evidence rather than indiscriminately supplying entire documents.

FR-010 — Question Answering

The system shall generate answers using retrieved policy evidence.

The system shall not intentionally use unsupported external knowledge to answer policy-specific questions.

FR-011 — Source Citations

Each evidence-based response shall provide source information sufficient for a user or reviewer to locate the supporting content.

At minimum, citations should identify:

- Document
 - Page
 - Section, where available
-

FR-012 — Product Comparison

The system shall support comparison of two or more insurance products.

Comparison criteria may include:

- Coverage
- Eligibility
- Exclusions
- Waiting periods
- Benefits
- Limitations
- Other policy-specific criteria available in the documents

The comparison shall distinguish between facts explicitly supported by source documents and conclusions derived from those facts.

FR-013 — User Requirement Extraction

For recommendation-oriented queries, the system shall identify relevant user requirements from the query.

Example:

Age: 45
Coverage requirement: Hospitalization
Preferred coverage: ₹10 lakh
Priority: Low waiting period

The extracted requirements shall be represented in a structured form before recommendation processing.

FR-014 — Agentic Query Decomposition

For complex queries, the system shall decompose the task into smaller operations.

Example:

User:
"Compare three policies for my requirements."

Agent:

1. Identify user requirements
2. Identify candidate policies
3. Retrieve eligibility
4. Retrieve coverage
5. Retrieve exclusions
6. Retrieve waiting periods
7. Compare evidence
8. Verify claims
9. Generate response

The exact workflow shall depend on query classification.

FR-015 — Evidence Verification

Generated policy-related claims shall be checked against available evidence where technically feasible.

The verification process shall determine whether a claim is:

- Supported
 - Contradicted
 - Insufficiently supported
-

FR-016 — Unsupported Claim Handling

If a generated claim cannot be adequately supported by the available evidence, the system shall not present it as an established policy fact.

Possible system actions:

1. Retrieve additional evidence.
 2. Re-evaluate the claim.
 3. Ask a clarification question.
 4. Abstain from making the claim.
-

FR-017 — Contradictory Evidence

If multiple policy passages provide conflicting information, the system shall identify the conflict instead of silently selecting an unsupported answer.

Where document version information exists, the system should prefer the applicable/current version according to configured rules.

FR-018 — Clarification

If the user has not provided enough information to perform a meaningful comparison or recommendation, the system shall ask a targeted clarification question.

Example:

"Which age group should be considered for the eligibility comparison?"

The system should avoid unnecessarily broad questions such as:

"Please provide more information."

FR-019 — Out-of-Domain Handling

The system shall identify queries that cannot be answered using the available insurance knowledge base.

Example:

"What will the stock market do tomorrow?"

The system should clearly state that the query is outside its supported knowledge domain.

7. Non-Functional Requirements

NFR-001 — Accuracy

The system shall be evaluated using predefined test queries and ground-truth answers.

NFR-002 — Grounding

Policy-specific claims should be traceable to retrieved source evidence.

NFR-003 — Explainability

Responses should provide understandable supporting evidence rather than unexplained recommendations.

NFR-004 — Latency

The system shall record end-to-end response latency.

Latency shall be measured separately where useful for:

- Query classification
 - Retrieval
 - Reranking
 - LLM generation
 - Verification
 - Total response time
-

NFR-005 — Cost Efficiency

The system shall record token consumption and, where applicable, estimated model/API cost.

NFR-006 — Reliability

The system shall handle:

- Empty retrieval results
- Malformed documents
- Unsupported queries
- LLM failures
- Vector-search failures
- Missing metadata
- Insufficient evidence

without silently producing misleading answers.

NFR-007 — Security

The system shall not expose confidential document content to unauthorized users.

API credentials and model keys shall be stored using environment-based configuration or a secure secrets mechanism.

NFR-008 — Maintainability

The implementation shall use modular components so that the following can be independently replaced or evaluated:

- Embedding model
 - Vector database
 - Retrieval strategy
 - Reranker
 - LLM
 - Agent workflow
 - Grounding validator
-

NFR-009 — Observability

The system shall log relevant execution information including:

- Query
- Query type
- Retrieved chunks
- Retrieval scores
- Agent steps
- Model latency
- Token usage
- Verification result
- Final response status

Sensitive information shall not be unnecessarily logged.

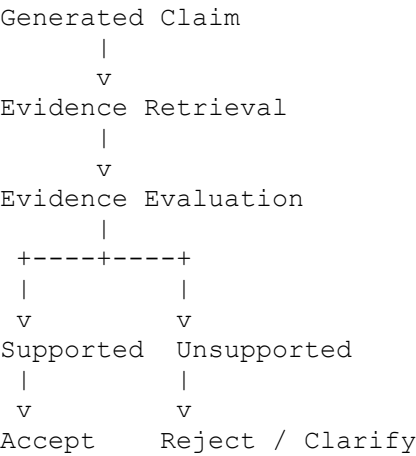
8. Grounding and Guardrail Requirements

Grounding is a core requirement rather than an optional feature.

The system shall follow this principle:

A policy-specific claim should be accepted only when sufficient evidence exists in the available knowledge base.

Conceptually:



The system shall distinguish:

Supported

The source explicitly supports the claim.

Contradicted

The source explicitly provides information inconsistent with the claim.

Insufficient Evidence

The source does not provide enough information to establish the claim.

This distinction is essential because **absence of evidence should not automatically be interpreted as evidence of absence.**

9. Recommendation Requirements

The system shall not present recommendations as absolute decisions.

A recommendation should contain:

1. User requirements considered
2. Policies evaluated
3. Relevant policy evidence
4. Comparison criteria
5. Advantages/disadvantages according to the documents
6. Limitations or missing information
7. Supporting citations

Example structure:

```
Recommendation
|
+-- User requirements
|
+-- Policy A evidence
|
+-- Policy B evidence
|
+-- Comparison
|
+-- Limitations
|
+-- Sources
```

10. Data Model — Initial Conceptual Model

The initial system may contain the following entities:

```
UserQuery
|
+---- QueryType
|
+---- UserRequirements
```

```

Document
|
+---- DocumentVersion
|
+---- Product
|
+---- Section
|
+---- Chunk
      |
      +---- Embedding
      |
      +---- Metadata

Query
|
+---- RetrievedChunk
|
+---- AgentExecution
|
+---- GeneratedClaim
      |
      +---- SupportingEvidence
      |
      +---- VerificationResult

Response
|
+---- Citation
|
+---- Recommendation

```

The final relational schema will be defined during technical design.

11. High-Level Agent Design

The agent architecture shall be modular.

Agent 1 — Query Analyzer

Responsibilities:

- Understand user intent
- Identify query type
- Extract user requirements
- Determine whether the query requires multiple steps

Agent 2 — Retrieval Planner

Responsibilities:

- Determine what information must be retrieved
 - Select appropriate retrieval operations
 - Generate sub-queries for complex tasks
-

Agent 3 — Evidence Retrieval Component

Responsibilities:

- Search the policy knowledge base
 - Retrieve relevant chunks
 - Apply metadata filters
 - Rank evidence
-

Agent 4 — Policy Analysis Agent

Responsibilities:

- Interpret retrieved policy clauses
 - Extract coverage
 - Identify exclusions
 - Identify eligibility conditions
 - Identify waiting periods
 - Compare policy-specific information
-

Agent 5 — Grounding Validator

Responsibilities:

- Inspect generated claims
 - Match claims to supporting evidence
 - Detect unsupported or contradictory claims
 - Mark claims for rejection or clarification
-

Agent 6 — Response Generator

Responsibilities:

- Produce the final user-facing response

- Include citations
 - Clearly communicate uncertainty
 - Avoid unsupported claims
-

12. Example End-to-End Workflow

User Query

"Compare Policy A and Policy B for hospitalization coverage and tell me which one has a shorter waiting period."

Step 1 — Query Analysis

Intent:
Product Comparison

Criteria:
1. Hospitalization coverage
2. Waiting period

Step 2 — Retrieval Planning

The system determines that it needs:

Policy A → hospitalization
Policy A → waiting period

Policy B → hospitalization
Policy B → waiting period

Step 3 — Retrieval

Relevant policy passages are retrieved.

Step 4 — Evidence Analysis

The system extracts structured facts.

Step 5 — Comparison

The facts are compared.

Step 6 — Grounding

Each generated comparison claim is checked against its source.

Step 7 — Response

The user receives:

Policy A:
Hospitalization: Supported
Waiting period: Source evidence available

Policy B:
Hospitalization: Supported
Waiting period: Source evidence available

Comparison:
...

Sources:
Policy A – Page X
Policy B – Page Y

If the waiting-period clause for one policy cannot be found, the system should explicitly state that instead of guessing.

13. Evaluation Strategy

The project shall not be evaluated solely through demonstrations.

A baseline shall be established before evaluating the enhanced architecture.

Baseline

```
User Query
  ↓
Basic Retrieval
  ↓
LLM
  ↓
Answer
```

Proposed System

```
User Query
  ↓
Classification
  ↓
Agent Planning
  ↓
Retrieval / Reranking
  ↓
Policy Analysis
  ↓
Grounding Verification
  ↓
Guardrails
  ↓
```

Answer

The two systems shall be compared.

14. Primary Evaluation Metrics

14.1 Retrieval Quality

Measure whether relevant policy passages are retrieved.

Possible metrics:

- Precision@K
 - Recall@K
 - MRR
 - nDCG
-

14.2 Answer Correctness

Measure whether the final answer correctly represents the ground-truth policy information.

14.3 Citation Accuracy

Measure whether cited passages actually support the claims made.

14.4 Unsupported Claim Rate

Measure the percentage of generated claims that lack sufficient evidence.

Unsupported Claim Rate =
$$\text{Unsupported Claims} / \text{Total Evaluated Claims}$$

14.5 Grounding/Faithfulness

Measure how consistently responses remain supported by retrieved evidence.

14.6 Abstention / Refusal Accuracy

Measure whether the system:

- Refuses when evidence is insufficient
- Answers when sufficient evidence exists

These should be reported separately.

14.7 Latency

Measure total response time and component-level latency.

14.8 Token / Cost Efficiency

Measure model token consumption and estimated API cost per query.

15. Failure Taxonomy

Every failure shall be classified rather than reported only as an overall failure percentage.

Initial categories:

F1 – Retrieval Failure
F2 – Missing Relevant Evidence
F3 – Incorrect Chunk Selection
F4 – Query Classification Failure
F5 – Agent Planning Failure
F6 – Reasoning Failure
F7 – Unsupported Generation
F8 – Citation Failure
F9 – Grounding Verification Failure
F10 – Incorrect Abstention
F11 – Unnecessary Abstention
F12 – Product Comparison Failure
F13 – Contradictory Evidence Handling Failure
F14 – Out-of-Domain Detection Failure

This taxonomy will be refined after the first evaluation cycle.

16. Testing Requirements

The system shall contain:

Unit Tests

For:

- Document processing
- Chunking
- Metadata extraction
- Retrieval
- Query classification
- Validators
- Citation generation

Integration Tests

For:

Query
→ Retrieval
→ Agent
→ LLM
→ Verification
→ Response

Evaluation Tests

A frozen evaluation dataset shall be established before extensive system tuning.

Test cases should include:

- Simple factual questions
- Multi-document questions
- Comparison questions
- Eligibility questions
- Exclusion questions
- Ambiguous questions
- Missing-evidence questions
- Contradictory-document cases
- Out-of-domain queries
- Adversarial/hallucination-inducing queries

17. Acceptance Criteria

The initial release shall satisfy the following:

AC-001

A supported insurance document can be ingested successfully.

AC-002

Document chunks retain traceable metadata.

AC-003

Relevant evidence can be retrieved for predefined evaluation queries.

AC-004

Responses contain source citations.

AC-005

The system can answer predefined policy questions using the knowledge base.

AC-006

The system can compare at least two supported insurance products.

AC-007

The system can execute a multi-step workflow for complex queries.

AC-008

Unsupported claims can be detected in predefined evaluation cases.

AC-009

The system can abstain or request clarification when sufficient evidence is unavailable.

AC-010

The enhanced system can be evaluated against the baseline RAG implementation.

AC-011

Latency and token/cost information are recorded during evaluation.

AC-012

Major system failures are classified using the failure taxonomy.

18. Constraints

The project shall initially use:

- Publicly available or synthetic insurance documents
- No confidential insurer information
- No unauthorized personal data
- Commercial or approved LLM APIs
- Laptop/cloud development infrastructure appropriate for the project

The system should remain modular enough to support alternative models and vector databases.

19. Risks

R1 — Hallucination

Risk: LLM generates unsupported insurance information.

Mitigation: Evidence grounding, citation verification, abstention.

R2 — Retrieval Failure

Risk: Relevant policy clause is not retrieved.

Mitigation: Hybrid retrieval, reranking, metadata filtering, evaluation of retrieval recall.

R3 — Ambiguous Policy Language

Risk: Policy clauses are difficult to interpret.

Mitigation: Preserve surrounding context, citations, structured analysis and human review.

R4 — Conflicting Documents

Risk: Different versions contain different information.

Mitigation: Document version metadata and explicit conflict handling.

R5 — Excessive Agent Complexity

Risk: Additional agents increase latency/cost without improving accuracy.

Mitigation: Baseline comparison and ablation experiments.

R6 — Recommendation Misinterpretation

Risk: Users interpret system output as professional insurance advice.

Mitigation: Clear system scope, evidence display and appropriate disclaimers.

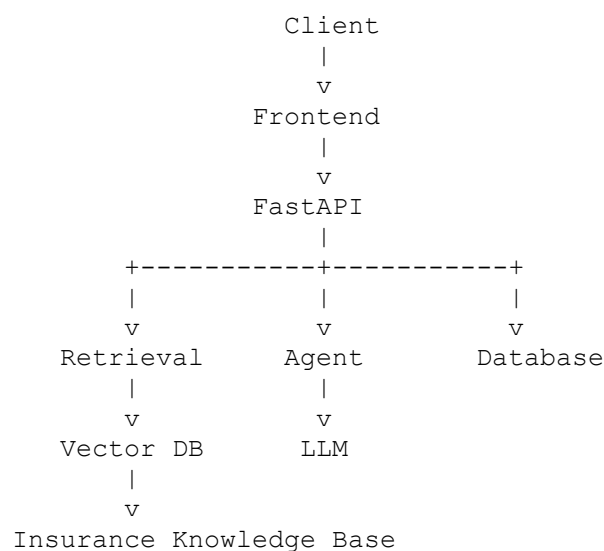
20. Security and Privacy

The system shall:

- Protect API credentials.
 - Avoid hardcoding secrets.
 - Restrict administrative document operations.
 - Avoid storing unnecessary personal information.
 - Log only required operational information.
 - Prevent unauthorized access to uploaded documents.
 - Clearly distinguish synthetic/public data from confidential data.
-

21. Deployment Concept

Initial deployment:



The system should eventually be packaged using Docker for reproducible deployment.

22. Development Phases

Phase 1 — Requirements & Dataset

- Freeze requirements
- Select document sources
- Define document schema
- Create evaluation dataset

Phase 2 — Baseline RAG

- Ingestion
- Chunking
- Embeddings
- Vector search
- Basic generation
- Citations

Phase 3 — Retrieval Improvement

- Metadata filtering
- Hybrid retrieval
- Reranking
- Retrieval evaluation

Phase 4 — Agentic Layer

- Query classification
- Requirement extraction
- Query planning
- Multi-step retrieval
- Policy comparison

Phase 5 — Grounding

- Claim extraction
- Evidence verification
- Unsupported-claim detection
- Abstention
- Clarification workflow

Phase 6 — Evaluation

- Baseline comparison

- Ablation experiments
- Failure taxonomy
- Accuracy
- Grounding
- Latency
- Cost

Phase 7 — Application Layer

- FastAPI
- User interface
- Authentication/access control where required
- Logging

Phase 8 — Deployment & Documentation

- Docker
- Deployment configuration
- API documentation
- User guide
- Technical design
- Final evaluation report

23. Definition of Done

The project shall not be considered complete merely because the chatbot produces answers.

The system is considered complete when:

- The baseline RAG system works.
- The proposed agentic workflow works.
- Complex insurance queries can be decomposed and processed.
- Product comparison works on the evaluation dataset.
- Responses contain traceable evidence.
- Unsupported claims are detected according to the implemented validation mechanism.
- The system can abstain or ask clarification when evidence is insufficient.
- Evaluation is performed against a frozen test set.
- Failure modes are documented.
- Latency and token/cost measurements are recorded.
- The implementation is reproducible.
- Technical and user documentation is complete.

24. Future Enhancements

Potential future extensions include:

- Multilingual insurance assistance
- OCR for scanned policy documents
- Voice-based interaction
- Structured policy knowledge graphs
- Advanced conflict resolution between policy versions
- Personalized policy analysis
- Enterprise document connectors
- Human expert review workflows
- Continuous evaluation
- Model/provider benchmarking

These features are outside the initial release unless explicitly incorporated into the project scope.

25. Document Change Control

Version	Status	Description
0.1	Draft	Initial requirements definition
0.2	Planned	Requirements review and refinement
1.0	Planned	Requirements baseline
1.x	Planned	Approved requirement changes

Any requirement added after the baseline should be documented as a controlled change rather than silently modifying the original scope.

26. Key Design Principle

The central design principle of the system is:

The assistant should optimize for evidence-supported usefulness rather than confident generation.

A successful response is therefore not necessarily the longest or most confident response.

A successful response is one that:

1. Understands the user's intent.
2. Retrieves relevant evidence.
3. Reasons over that evidence appropriately.
4. Distinguishes facts from conclusions.
5. Provides traceable sources.

6. Identifies insufficient evidence.
7. Avoids unsupported claims.
8. Requests clarification when necessary.

This principle will guide both system implementation and evaluation.